Contest Platform with Live Leaderboard

Walkthrough Manual, Database Architecture & Database Viva Voce Reference

Team	Upayan Mazumder (24BDS0367), Aditya Rawat (24BCE2992), Shloak Sinha (24BDS0378)
Course	BCSE302P – Database Systems Lab
Component	Review 3 – Operator Manual & Comprehensive Database Viva Defense
Repository	github.com/upayanmazumder/rankstack

1. System Architecture & Data Flow

Rankstack separates persistent document storage from ephemeral caching to handle contest workloads:

- **MongoDB Replica Set (`rs0`):** System of record. Houses five collections with `$jsonSchema` validation. Runs as a replica set to support multi-document ACID transactions.
- **Redis Ephemeral Store:** In-memory key-value store for live leaderboards (sorted sets), opaque user sessions (with TTL), and submission rate limiters.
- **FastAPI Backend (`api/`):** Implements REST endpoints, session checks, and multi-document transactions using synchronous PyMongo.
- **Next.js 16 Web Frontend (`app/`):** Client application using App Router, TanStack Query, and Zustand stores. Proxies session cookies to backend Bearer tokens.

Submission Data Flow

1. The participant submits a solution via `POST /submissions`.
2. The API checks the user rate limit counter in Redis (key: `rate_limit:<userId>`).
3. The backend executes a MongoDB transaction: inserts the submission document and increments `problems.attemptCount` atomically.
4. An evaluation update sets submission status and increments score totals on the user or team document.
5. The route updates the Redis leaderboard sorted set using `ZINCRBY leaderboard:<contestId>`.
6. Clients poll `GET /contests/{id}/leaderboard` every 30 seconds to refresh the standings.

2. How to Run the Project

Open the repository in two terminals. Run these commands from the repository root:

```
# Terminal 1: Database & Backend Server
make up                  # Start MongoDB (rs0) and Redis in Docker
make init-db             # Create collections, $jsonSchema rules, and indexes
make seed                # Wipe and seed 112 verified sample records
make api                 # Start FastAPI at http://localhost:8000

# Terminal 2: Web Client
cd app && pnpm dev --port 3000   # Start Next.js client at http://localhost:3000
```

3. Step-by-Step Demonstration Sequence

1. **Public Cloud Deployment:** Open `rankstack.upayan.dev` to show the live web deployment.
2. **User Login:** Open `http://localhost:3000/login`. Sign in as `user1@rankstack.io` with password `Passw0rd!`.
3. **Contests Catalog:** Browse contests. Show search filtering and status tabs (All, Live, Upcoming, Ended). Open the live contest.
4. **Problem Workspace:** Click Problems. Choose a problem, enter or select an answer, and submit. Show the attempt count increment.
5. **Live Leaderboard:** Click Leaderboard. Show real-time participant rankings, submission counts, and scores fetched from Redis.
6. **Teams Catalog:** Click Teams. Show score-ranked squads. Open a team profile to inspect member cards and combined score.

7. **Admin Dashboard:** Navigate to `/admin`. Show contest and problem management and subjective submission review.
8. **Automated Script Verification:** Run `make demo` in the terminal. This verifies database connectivity, executes full user CRUD operations, demonstrates `IXSCAN` index scans via `explain()`, tests an ACID transaction, runs all five aggregation pipelines, and checks Redis TTLs.

4. Comprehensive Database Viva Voce Questions & Answers

Examiner questions in Review 3 will focus on database architecture, schema decisions, transaction guarantees, aggregation stages, and cache consistency. Study these technical answers:

Examiner Question	Detailed Database Answer
1. Why use MongoDB instead of relational SQL (MySQL / Postgres)?	Contest problems have polymorphous structures: MCQs need arrays of string options and a correct answer; coding challenges store test cases with input/output pairs and constraints; subjective questions require evaluation rubrics and word limits. In a relational database, this requires multiple sparse tables or EAV anti-patterns with complex joins. MongoDB documents store each problem type naturally as a single document with type-specific fields validated by `$jsonSchema`.
2. Explain your data model. Did you embed or reference relationships?	We used a **hybrid modeling approach** based on data access patterns: • **Referencing:** Problems reference `contestId`. Submissions reference `contestId`, `problemId`, and `submittedBy.refId`. Teams reference user IDs in `memberIds`. This prevents unbound document growth and avoids duplicating problem statements or participant profiles across submissions. • **Embedding:** Contest participants are embedded as an array of `{ refType, refId }` subdocuments inside the contest document. Coding problem test cases and MCQ option lists are embedded inside problem documents because they are always read together with the problem.
3. How does MongoDB enforce schema validation?	Validation is enforced at the database level using MongoDB's `$jsonSchema` operator inside `createCollection` (configured in `api/db.py`). If an insert or update violates field constraints (e.g., missing required fields, invalid problem type string, negative attempt counts), the storage engine rejects the write with a `DocumentValidationFailure` error. This guarantees data integrity even if an external script writes directly to the database.
4. Why is a replica set (`rs0`) required even on a single machine?	MongoDB only supports multi-document transactions (via `client.start_session()` and `session.start_transaction()`) on a replica set or sharded cluster. In standalone mode, MongoDB does not write to the replication operations log (oplog), which is necessary for multi-document rollback and isolation. Attempting transactions on a standalone instance raises an explicit error: *"Transaction numbers are only allowed on a replica set member or mongos"*.
5. Walk me through a multi-document ACID transaction in your code.	In `api/services.py:create_submission`: 1. Start a client session: `with client.start_session() as session:` 2. Start transaction block: `with session.start_transaction():` 3. Insert the submission document into `db.submissions` passing `session=session`. 4. Atomically increment `attemptCount` in `db.problems` using `$inc: {"attemptCount": 1}` passing `session=session`. 5. If either write fails, the entire block aborts and rolls back, ensuring attempt counts never desynchronize from submission records.
6. What indexes did you build, and what query plans do they produce?	We configured four custom B-tree indexes: • `users.email` (unique): Prevents duplicate registrations and gives `O(log N)` login lookup. • `problems.contestId`: Index for fetching all problems in a contest. • `submissions (contestId, submittedBy.refId)`: Compound index for retrieving a user's contest submission history. • `teams.memberIds`: Multikey index for finding which teams a user belongs to. Running `.explain("executionStats")` confirms an **`IXSCAN`** stage followed by **`FETCH`**, completely avoiding collection scans (**`COLLSCAN`**).
7. What is a multikey index and where did you use it?	A multikey index indexes array fields by creating separate index keys for every element in the array. We used a multikey index on `teams.memberIds`, where each team document contains an array of user ObjectIds. This allows MongoDB to query `{"memberIds": userId}` using an index scan to instantly find all teams a user belongs to.

8. Explain the pipeline stages in your 5 aggregation queries.	Located in `api/aggregations/`: 1. **Leaderboard:** `$match` (by contestId) → `$group` (by `submittedBy.refId`, sum points) → `$sort` (descending score) → `$setWindowFields` (computes `$denseRank`). 2. **Average Score by Difficulty:** `$lookup` (joins problems on `problemId`) → `$unwind` → `$group` (by `problems.difficulty`, computes `$avg` score). 3. **Submission Status by Contest:** `$group` (by contestId and status, counts total) → `$group` (by contestId, pushes status counts into an array). 4. **Distinct Problems Solved:** `$match` (status="correct") → `$group` (by user and problemId) → `$group` (by user, counts distinct problemIds). 5. **Team Performance:** `$lookup` (joins submissions matching team `memberIds`) → computes team member count, submission count, and total score.
9. Why use Redis alongside MongoDB?	• **Live Leaderboard:** Stored in Redis sorted sets (`ZSET`). Reading top 100 participants via `ZREVRANGE` takes sub-millisecond memory time, preventing expensive analytical aggregation queries on every browser refresh. • **Session Expiration:** Session tokens are stored with a 3600-second TTL. Redis automatically evicts expired sessions without needing scheduled cleanup cron jobs. • **Rate Limiting:** Atomic increments (`INCR`) with 60-second expiration enforce a 10-submission limit per window without writing ephemeral counters to disk.
10. How do you handle cache invalidation and Redis failover?	We implement a write-through invalidation pattern tracked in MongoDB via the `dirty_leaderboards` collection. Every scored submission increments a version counter. If Redis restarts or drops the sorted set, the API detects the missing cache, runs the aggregation pipeline to rebuild the sorted set, and reconciles the version counter.
11. How do you prevent SQL / NoSQL injection attacks?	We avoid raw query string interpolation. All user inputs are parsed and validated through Pydantic schemas. ObjectIds are validated and converted using `bson.ObjectId()`, which rejects malformed operator injections (such as `{"$gt": ""}`) and raises a 400 Bad Request error.

5. Operator Manual & Role Access Rules

Participant Workflow

1. Register an account or sign in at `/login`.
2. Browse contests. Click **Join contest** before the start deadline.
3. Open the contest during the live schedule and solve problems in the workspace.
4. Review evaluated answers under Submissions and inspect standings on the live leaderboard.

Administrator Workflow

1. Sign in with an account having role `admin` (e.g. `user1@rankstack.io`).
2. Open `/admin` to create or edit contest problems and test cases.
3. Review pending subjective submissions and assign scores. Score updates trigger atomic score recalculations and update Redis leaderboards automatically.

> **Access Rule:** Participant accounts are blocked from accessing `/admin` routes and redirect to `/contests` automatically via Next.js proxy middleware.